

NNGraph DSL Compiler

A Domain-Specific Language for Describing Neural Network Architectures
and Automatic PyTorch Code Generation

Project Technical Report

July 1, 2026

Contents

Abstract	2
1 Introduction	3
1.1 Motivation	3
1.2 Goal	4
1.3 Scope	4
2 Related Work	5
2.1 The NADER Framework	5
2.2 ANTLR4	5
2.2.1 Listener Pattern vs. Visitor Pattern	5
2.3 AST Utilities and Visualizations	6
2.3.1 Why AST Is Not Used Here	6
3 Language Design	7
3.1 Program Structure	7
3.2 Grammar	8
3.3 Type System	9
3.4 Layer Types	9
3.4.1 Parameterized Layers	9
3.4.2 Activations	10
3.4.3 Graph Operations (No Module)	10
3.4.4 Universal Parameters	11
3.5 Edge Syntax	11
3.6 Complete Example: MLP	11
4 Implementation	14
4.1 Compiler Architecture	14
4.1.1 File Responsibilities	15
4.2 Semantic Analyzer	15
4.2.1 Value Propagation in the Tree	15

4.2.2	Validation Rules	16
4.2.3	Error Message Format	16
4.2.4	_FakeCtx for Post-Walk Errors	17
4.2.5	Listener Execution Flow	17
4.3	Code Generator	22
4.4	Variable Assignment Algorithm	22
4.4.1	MLP Trace (Linear Chain)	23
4.4.2	ResBlock Trace (Fan-Out Then Merge)	23
4.4.3	InceptionBlock Trace (Two Parallel Branches)	24
4.4.4	TransformerEncoder Trace (Two Implicit Residuals)	24
4.4.5	Special Code Generation Cases	25
4.4.6	Output Code Structure	26
4.5	Shape Inference	26
4.6	Graphviz Visualization	27
4.7	Training Scaffold	27
4.8	Quantization Annotations	27
4.9	ONNX Export	27
5	Evaluation	29
5.1	Test Suite	29
5.2	Forward Pass Results	30
5.3	Variable Correctness Proof	30
5.4	Codebase Statistics	31
6	Design Decisions	32
6.1	Listener vs. Visitor	32
6.2	load_from_listener() vs. generate(traversal)	32
6.3	Variable Naming with node_id	32
7	Conclusion and Future Work	33
7.1	Conclusion	33
7.2	Implemented Features	33
7.3	Future Work	33
	References	34

Abstract

nngraph dsl is a domain-specific language we made to describe neural networks as directed graphs. it takes `.nng` files and spits out working PyTorch code, giving you a full `nn.Module` with `__init__` and `forward` methods ready to go

we built the compiler using ANTLR4 version 4.13.2 and went with the listener pattern for the semantic analysis part. we use Kahn's topological sort for both finding cycles in the semantic analyzer and figuring out the execution order in the code generator. on top of that, it supports tensor shape inference, graphviz visualizations, training scaffolding, quantization annotations, and even ONNX export

our test suite has 55 tests spread over four files. we tested it on four main architectures—MLP, ResBlock, InceptionBlock, and TransformerEncoder—and they all compile fine and get the right output tensor shapes in their forward pass

Introduction

1.1 Motivation

writing `nn.Module` subclasses in PyTorch by hand for complicated architectures gets super repetitive and it's easy to mess up. if you have a network with ten layers, you gotta:

- define each layer one by one with the right params in `__init__`
- handle the execution order manually in `forward`
- come up with names for intermediate variables when the graph branches
- make sure you don't overwrite a shared tensor before another layer consumes it

take a look at this PyTorch code you'd normally have to write manually:

Listing 1.1.1 ▶ Manual boilerplate for a ResBlock

```
1 class ResBlock(nn.Module):
2     def __init__(self):
3         super(ResBlock, self).__init__()
4         self.conv1 = nn.Conv2d(
5             in_channels=64, out_channels=64,
6             kernel_size=3, stride=1, padding=1)
7         self.bn1 = nn.BatchNorm2d(num_features=64)
8         self.relu1 = nn.ReLU()
9         # ... and so on for each layer
10
11     def forward(self, x):
12         identity = x
13         x = self.conv1(x)
14         x = self.bn1(x)
15         x = self.relu1(x)
16         # ... manage residual connection manually
```

```
17         x = x + identity
18     return x
```

with `nngraph dsl`, you just describe the architecture once as a graph and our compiler generates all that code for you

1.2 Goal

the whole point of `nngraph dsl` is to make the `.nng` file the single source of truth. you just describe the graph, and our compiler handles all the boilerplate

1.3 Scope

we built this tool for static architectures, meaning networks where you know the computational graph right at definition time. we're planning to add support for dynamic control flow inside **`forward`** in future versions

Related Work

2.1 The NADER Framework

Definition 2.1.1 ▶ NADER

NADER (Neural Architecture Design via Representation) is a framework that models neural network architectures as directed computational graphs (DAGs). Each node represents a layer operation and each edge represents tensor flow

we based our language design directly on this graph model

2.2 ANTLR4

Definition 2.2.1 ▶ ANTLR4

ANTLR4 (ANother Tool for Language Recognition) is a tool that automatically generates a lexer, parser, and listener/visitor from a `.g4` grammar file. Version used: ANTLR 4.13.2

2.2.1 Listener Pattern vs. Visitor Pattern

ANTLR4 gives us two ways to traverse the parse tree:

- **Listener:** `ParseTreeWalker` just walks the tree for you and calls `enter*/exit*` methods, so you don't need to return values
- **Visitor:** you control the traversal yourself and have to return a value from each method

we went with the listener pattern for our compiler. we use the `exit*` methods in `custom_listener.py` to build up state in `self.nodes` and `self.edges`—it just felt more natural for extracting a graph from a parse tree

2.3 AST Utilities and Visualizations

we added some abstract syntax tree (AST) stuff to help with debugging and visualization if you want to inspect the structure. we used three files for this:

- `ast.py` — has the **AST** and **TreeNode** classes
- `make_ast_subtree.py` — handles merging parse tree nodes
- `ast_to_networkx_graph.py` — for optional graph visualizations

2.3.1 Why AST Is Not Used Here

normal compilers usually represent code as an abstract syntax tree (AST) because expression-based languages have operator precedence and nested scopes that map nicely to trees

but `nngraph dsl` describes neural network architectures as *directed acyclic graphs* (DAGs). the nodes are operations and the edges are the flow of tensors. the graph itself is our domain model, not the parse tree

our compiler has to do a bunch of graph-heavy stuff that needs adjacency lists and explicit edges:

- **topological sort** (using Kahn’s algorithm) to figure out execution order in `forward()`
- **cycle detection** using the top-sort to make sure it’s a valid DAG
- **reachability analysis** using BFS to check paths from input to output
- **residual connection validation** to make sure merge nodes have the right in-degree
- **shape inference** to push tensor shapes across the edges
- **Graphviz DOT export** so we can draw the architecture
- **ONNX export** since ONNX uses graphs too

if we used an AST, it would just encode the parser rules like `model_block`, `graph_block`, and `node_decl`, which kind of hides the actual computational graph we care about. so instead, our semantic analyzer directly dumps nodes and edges into dictionaries and lists since that’s what graph algorithms actually want

we kept the AST building code (`exitEveryRule`, `exitStart`) in `custom_listener.py` for debugging with `--show-ast`, but we completely ignore it during normal compilation

3.1 Program Structure

every `.nng` file has up to three blocks, and the `config` block is completely optional

Listing 3.1.1 ▶ General structure of a `.nng` file

```

1  model <Name> {
2      input  <id> : tensor(<shape>)
3      output <id>
4  }
5
6  graph {
7      node <id> : <LayerType>(<params>)
8      ...
9      edge <src> -> <dst>
10     edge <src> -> <dst> [label="<string>"]
11     ...
12 }
13
14 config {
15     batch_size = <int>
16     device      = "cpu" | "cuda"
17     loss        = "<LossFn>"           // enables training
18     ⇨ scaffold
19     optimizer   = "<Optimizer>"       // Adam | SGD | AdamW |
20     ⇨ RMSprop
21     lr          = <float>              // learning rate
22     epochs      = <int>                // training epochs
23 }
```

Definition 3.1.2 ▶ Model block

we use the **model** block to define the generated class name, along with what the input and output node IDs and tensor shape look like

Definition 3.1.3 ▶ Graph block

the **graph** block is where you define all the layer nodes and how they connect with directed edges. it doesn't matter what order you declare the edges in since our compiler figures out the execution order automatically from the structure

Definition 3.1.4 ▶ Config block

the **config** block lets you tweak stuff in the `__main__` section of the output code. we default **batch_size** to 1 and **device** to `'cpu'`. if you set a **loss** function, the code generator will actually spin up a full training loop with your optimizer and loss built right in

3.2 Grammar

Definition 3.2.1 ▶ NNGraph.g4 statistics

- grammar file: **NNGraph.g4** (56 lines)
- parser rules: 14 rules (snake_case)
- lexer tokens: 20 tokens (ALL_CAPS)

here are our full grammar rules:

Listing 3.2.2 ▶ NNGraph.g4 grammar

```

1  grammar NNGraph;
2  start      : model_block graph_block config_block? EOF;
3  model_block : MODEL ID '{' input_decl output_decl '}';
4  input_decl  : INPUT ID ':' TENSOR shape_expr;
5  output_decl : OUTPUT ID;
6  graph_block : GRAPH '{' (node_decl | edge_decl)* '}';
7  node_decl   : NODE ID ':' layer_expr;
8  layer_expr  : ID '(' param_list? ')';

```

```

9  param_list  : param (',' param)*;
10 param       : ID '=' value;
11 edge_decl   : EDGE ID ARROW ID ('[' LABEL '=' STRING ']')?;
12 config_block: CONFIG '{' config_entry* '}';
13 config_entry: ID '=' value;
14 value       : FLOAT_LITERAL | INT_LITERAL | BOOL_LITERAL
15             | STRING | NONE | shape_expr;
16 shape_expr  : '(' INT_LITERAL (',' INT_LITERAL)* ')';

```

important design note: `input_decl` uses `TENSOR shape_expr` instead of `TENSOR '(' shape_expr ')'` because `shape_expr` already includes the parens. we messed this up in earlier versions and had to fix it

3.3 Type System

DSL Type	Python Type	Example
<code>int</code>	<code>int</code>	<code>128</code>
<code>float</code>	<code>float</code>	<code>0.1</code>
<code>bool</code>	<code>bool</code>	<code>true / false</code>
<code>string</code>	<code>str</code>	<code>"relu"</code>
<code>shape</code>	<code>tuple[int]</code>	<code>(64, 32, 32)</code>
<code>None</code>	<code>NoneType</code>	<code>None</code>

we check the difference between `int` and `float` at compile time, so if you try to write `Dropout(p=1)`, the compiler will throw an error since `p` needs a `float` and not an `int`

3.4 Layer Types

3.4.1 Parameterized Layers

these layers will generate a `self.<id> = nn.X(...)` line right inside the `__init__` method

DSL Keyword	PyTorch Class	Key Parameters
Linear	<code>nn.Linear</code>	<code>in_features</code> , <code>out_features</code> , <code>bias</code>
Conv2d	<code>nn.Conv2d</code>	<code>in_ch</code> , <code>out_ch</code> , <code>kernel</code> , <code>stride</code> , <code>padding</code>
Conv1d	<code>nn.Conv1d</code>	<code>in_ch</code> , <code>out_ch</code> , <code>kernel</code> , <code>stride</code>
BatchNorm2d	<code>nn.BatchNorm2d</code>	<code>num_features</code>
LayerNorm	<code>nn.LayerNorm</code>	<code>normalized_shape</code>
MaxPool2d	<code>nn.MaxPool2d</code>	<code>kernel</code> , <code>stride</code>
AvgPool2d	<code>nn.AvgPool2d</code>	<code>kernel</code> , <code>stride</code>
Dropout	<code>nn.Dropout</code>	<code>p</code>
Flatten	<code>nn.Flatten</code>	<code>start_dim</code> , <code>end_dim</code>
Embedding	<code>nn.Embedding</code>	<code>num_embeddings</code> , <code>embedding_dim</code>
MultiHeadAttn	<code>nn.MultiheadAttention</code>	<code>embed_dim</code> , <code>num_heads</code>
LSTM	<code>nn.LSTM</code>	<code>input_size</code> , <code>hidden_size</code> , <code>num_layers</code>
GRU	<code>nn.GRU</code>	<code>input_size</code> , <code>hidden_size</code>

3.4.2 Activations

`ReLU`, `Sigmoid`, `Tanh`, `GELU`, `Softmax(dim)`, `LeakyReLU(negative_slope)`, `ELU(alpha)`

3.4.3 Graph Operations (No Module)

these ones don't generate any code in `__init__` because they're just operations on the graph:

Keyword	Parameters	Generated Code
Add	—	<code>out = a + b</code>
Concat	<code>dim: int</code>	<code>out = torch.cat([a,b], dim=d)</code>
Residual	—	<code>out = main + shortcut</code>
Split	<code>chunks: int, dim: int</code>	<code>split_0, split_1, ... = torch.chunk(x,n,dim=d)</code>

3.4.4 Universal Parameters

you can slap the **quant** parameter on any node no matter the layer type. we support "dynamic", "static", and "qat" for this

Listing 3.4.1 ▶ Quantization annotation example

```
1 node fc1 : Linear(in_features=784, out_features=128,
  ↪ quant="dynamic")
```

3.5 Edge Syntax

Definition 3.5.1 ▶ Simple edge

```
edge src -> dst
```

defines a simple directed edge pointing from **src** to **dst**

Definition 3.5.2 ▶ Labeled edge

```
edge src -> dst [label="name"]
```

defines an edge with a string label attached. we use labels for two main things:

- "shortcut": tags the skip connection edge feeding into a **Residual()** node
- "residual_*": tags a skip connection going into a node that takes two inputs, which happens a lot in Transformers

3.6 Complete Example: MLP

here's what the **.nng** file looks like for a standard three-layer network:

Listing 3.6.1 ▶ mlp.nng

```
1 model MLP {
2     input x : tensor(784)
3     output out
4 }
5
6 graph {
7     node fc1 : Linear(in_features=784, out_features=256)
8     node relu1 : ReLU()
9     node fc2 : Linear(in_features=256, out_features=128)
10    node relu2 : ReLU()
11    node drop : Dropout(p=0.3)
12    node fc3 : Linear(in_features=128, out_features=10)
13    node out : Softmax(dim=1)
14
15    edge x -> fc1
16    edge fc1 -> relu1
17    edge relu1 -> fc2
18    edge fc2 -> relu2
19    edge relu2 -> drop
20    edge drop -> fc3
21    edge fc3 -> out
22 }
23
24 config {
25     batch_size = 64
26     device = "cuda"
27 }
```

and here's the PyTorch code it spits out:

Listing 3.6.2 ▶ Generated PyTorch code for MLP

```
1 import torch
2 import torch.nn as nn
3
```

```
4
5 class MLP(nn.Module):
6     def __init__(self):
7         super(MLP, self).__init__()
8         self.fc1 = nn.Linear(in_features=784,
9                               ↪ out_features=256)
10        self.relu1 = nn.ReLU()
11        self.fc2 = nn.Linear(in_features=256,
12                              ↪ out_features=128)
13        self.relu2 = nn.ReLU()
14        self.drop = nn.Dropout(p=0.3)
15        self.fc3 = nn.Linear(in_features=128,
16                              ↪ out_features=10)
17        self.out = nn.Softmax(dim=1)
18
19    def forward(self, x):
20        x = self.fc1(x)
21        x = self.relu1(x)
22        x = self.fc2(x)
23        x = self.relu2(x)
24        x = self.drop(x)
25        x = self.fc3(x)
26        x = self.out(x)
27        return x
28
29 if __name__ == '__main__':
30     device = torch.device('cuda')
31     model = MLP().to(device)
32     x = torch.randn(64, 784).to(device)
33     print(model(x).shape)
```

Implementation

4.1 Compiler Architecture

Definition 4.1.1 ▶ Compilation pipeline

our compiler runs through six stages:

1. **parsing**: `FileStream` $\rightarrow$ `NNGraphLexer` $\rightarrow$ `NNGraphParser.program()`
 $\rightarrow$ parse tree
2. **semantic analysis**: `ParseTreeWalker` + `NNGraphCustomListener` $\rightarrow$
graph model
3. **topological sort**: uses Kahn's algorithm to compute the execution order
4. **shape inference**: pushes tensor shapes through the graph and complains if
dimensions don't match (you can see this with `--show-shapes`)
5. **code generation**: `CodeGenerator.generate()` $\rightarrow$ Python string
6. **output**: writes out a `.py` file, Graphviz DOT output, or an ONNX export

4.1.1 File Responsibilities

File	Role
NNGraph.g4	grammar source of truth (56 lines)
gen/NNGraphLexer.py	generated by ANTLR4
gen/NNGraphParser.py	generated by ANTLR4
gen/NNGraphListener.py	base listener — shouldn't be edited
custom_listener.py	semantic analysis
code_generator.py	PyTorch code generation
shape_inference.py	compile-time tensor shape inference
graph_viz.py	Graphviz DOT visualization
onnx_export.py	ONNX model export
main.py	CLI interface and <code>compile_nng()</code>

4.2 Semantic Analyzer

Definition 4.2.1 ▶ NNGraphCustomListener

we put all the semantic analysis logic in `NNGraphCustomListener(NNGraphListener)` inside `custom_listener.py`. here's what we keep track of:

- `self.nodes`: dict[id, {type, params, line}]
- `self.edges`: list[{src, dst, label, line}]
- `self.config`: dict[key, value]

4.2.1 Value Propagation in the Tree

the `exitShape_expr` and `exitValue` methods attach their values right onto the context object:

Listing 4.2.2 ▶ Value propagation via `ctx.pvalue` in `exitShape_expr`

```

1 def exitShape_expr(self, ctx):
2     ints = [int(ctx.INT_LITERAL(i).getText())
```

```

3         for i in range(ctx.INT_LITERAL().__len__())
4             ctx.pvalue = tuple(ints)
5             ctx.ptype  = tuple

```

doing it this way makes sure every context has access to its children's values once it finishes

4.2.2 Validation Rules

we run eight validation rules split into two phases

phase one (during tree walk):

1. **duplicate node ID**: caught in `exitNode_decl`
2. **invalid edge reference**: caught in `exitEdge_decl`
3. **unknown layer type**: caught in `exitNode_decl`
4. **parameter type mismatch**: caught in `exitNode_decl`

phase two (in `exitGraph_block`, when the graph is done):

5. **undeclared output node**: checked in `_validate_output_declared`
6. **orphan node**: checked in `_validate_orphans`
7. **residual arity $\neq 2$** : checked in `_validate_residual_arity`
8. **cycle detection via Kahn's algorithm**: checked in `_validate_no_cycles`
9. **BFS reachability**: checked in `_validate_reachability`

4.2.3 Error Message Format

Listing 4.2.3 ▶ Error format

```

1 [line L:C] SemanticError: <message>
2 [line L] ShapeError at '<node>': <message>

```

L is the 1-based line number and **C** is the 0-based token column. if any of these errors pop up, we kill the compilation immediately and exit with code 1

4.2.4 `_FakeCtx` for Post-Walk Errors

errors that happen in `exitGraph_block` don't actually have a live context tied to them. so we use an internal `_FakeCtx` class just to carry over the line number we saved in the node's dictionary:

Listing 4.2.4 ▶ `_FakeCtx` class for preserving line numbers

```
1 class _FakeCtx:
2     class start:
3         line    = info["line"]
4         column  = 0
```

4.2.5 Listener Execution Flow

`ParseTreeWalker.walk(listener, parse_tree)` runs a depth-first traversal of the tree. for every single node, it calls `enterX(ctx)` before visiting the kids and `exitX(ctx)` after they're all done

Definition 4.2.5 ▶ Walker invocation in `main.py`

```
1 listener = NNGraphCustomListener()
2 listener.rule_names = parser.ruleNames
3 walker = ParseTreeWalker()
4 walker.walk(listener, parse_tree)
```

our custom listener only overrides the `exit*` methods, so all the `enter*` methods just stay as empty no-ops from the base `NNGraphListener`

Attribute Propagation (Bottom-Up)

values move up the tree through context attributes like this:

Listing 4.2.6 ▶ `exitValue` stores parsed value in `ctx.pvalue`

```
1 def exitValue(self, ctx):
2     if ctx.FLOAT_LITERAL():
3         ctx.pvalue = float(ctx.FLOAT_LITERAL().getText())
4         ctx.ptype  = float
```

```

5     elif ctx.INT_LITERAL():
6         ctx.pvalue = int(ctx.INT_LITERAL().getText())
7         ctx.ptype  = int
8     elif ctx.BOOL_LITERAL():
9         ctx.pvalue = ctx.BOOL_LITERAL().getText() == 'true'
10        ctx.ptype  = bool
11    # ... other cases

```

then parent rules just read the attributes from their kids:

Listing 4.2.7 ▶ `exitParam` reads `ctx.value().pvalue` from child

```

1  def exitParam(self, ctx):
2      ctx.param_name = ctx.ID().getText()
3      ctx.param_value = ctx.value().pvalue
4      ctx.param_type  = ctx.value().ptype

```

Semantic Extraction and Detailed Method Behavior

we wrote a bunch of `exit*` methods to grab info from the tree bottom-up. here's exactly what each listener function is doing under the hood:

- **`exitShape_expr(ctx)`**: grabs the dimensions from `INT_LITERAL` tokens, sticks them in a Python `tuple`, and attaches it to `ctx.pvalue` with `ctx.ptype` set to `tuple`
- **`exitValue(ctx)`**: parses literals like `FLOAT_LITERAL`, `INT_LITERAL`, `BOOL_LITERAL`, `STRING`, `NONE`, or nested shapes. it turns the text into actual Python types and saves them to `ctx.pvalue`
- **`exitModel_block(ctx)`**: grabs the model's name via `ctx.ID().getText()` and saves it in `self.model_name`
- **`exitInput_decl(ctx)`**: saves the input variable ID in `self.input_id` and its shape in `self.input_shape`. it also drops a fake `"__input__"` node into `self.nodes` at the current line number so we can validate edges starting from the input later on
- **`exitOutput_decl(ctx)`**: saves the output variable ID in `self.output_id` so we know where the network ends

- **exitParam(ctx)**: handles key-value pairs for parameters by mapping the ID to `ctx.param_name` and pulling the parsed values into `ctx.param_value`
- **exitLayer_expr(ctx)**: processes layer setups by matching the class name from the ID and building a dictionary of the parameters, which then gets stuck to `ctx.params`
- **exitNode_decl(ctx)**: registers nodes and checks for basic errors. it makes sure you haven't declared the same ID twice, checks the layer against our `LAYER_SCHEMA` to ensure params have the right types, validates any `quant` annotations, and finally adds the node to `self.nodes`
- **exitEdge_decl(ctx)**: deals with connections. it pulls the source and destination IDs, makes sure they both actually exist in `self.nodes`, processes any labels like "shortcut", and records the edge in `self.edges`
- **exitConfig_entry(ctx)**: grabs config overrides like `device` or `batch_size` and puts them into `self.config`
- **exitGraph_block(ctx)**: fires after we've walked the whole graph block. it runs five separate validation checks: makes sure the output node exists, checks for orphaned nodes, verifies residual node inputs, looks for cycles, and ensures there's a valid path from input to output
- **exitEveryRule(ctx)**: fires every time a rule finishes. if we loaded rule names, it calls `make_ast_subtree` to build and merge AST nodes bottom-up
- **exitStart(ctx)**: wraps up AST construction by tying the whole hierarchy to the listener's root collector

Graph Validation Algorithms

`exitGraph_block` runs five validation passes once the graph is completely built:

1. orphan detection (`_validate_orphans`):

Listing 4.2.8 ▶ Orphan detection algorithm

```

1 referenced = set()
2 for e in self.edges:
3     referenced.add(e["src"])
4     referenced.add(e["dst"])

```

```

5  for node_id, info in self.nodes.items():
6      if info["type"] == "__input__":
7          continue
8      if node_id not in referenced:
9          error(f"Node '{node_id}' is declared but never
              ↪ referenced.")

```

we gather all the node IDs mentioned in edges. if we find a declared node (other than the input) that isn't in that set, it's an orphan—declared but never connected to anything

2. cycle detection (`_validate_no_cycles`):

we use Kahn's topological sort for this. if you can fully sort a DAG, you're good. but if there are still nodes left with `in_degree > 0` at the end, you've got a cycle

Listing 4.2.9 ▶ Kahn's algorithm for cycle detection

```

1  in_degree = dict.fromkeys(self.nodes, 0)
2  adj       = {n: [] for n in self.nodes}
3  for e in self.edges:
4      adj[e["src"]].append(e["dst"])
5      in_degree[e["dst"]] += 1
6
7  queue = deque(n for n, d in in_degree.items() if d == 0)
8  visited = 0
9  while queue:
10     node = queue.popleft()
11     visited += 1
12     for nxt in adj[node]:
13         in_degree[nxt] -= 1
14         if in_degree[nxt] == 0:
15             queue.append(nxt)
16
17  if visited != len(self.nodes):
18     remaining = [n for n, d in in_degree.items() if d > 0]
19     error(f"Cycle detected involving nodes: {remaining}.")

```

3. reachability analysis (`_validate_reachability`):

we do a quick BFS starting from the input node to make sure every declared node and the output node are actually reachable

Listing 4.2.10 ▶ BFS reachability check

```

1  adj = {n: [] for n in self.nodes}
2  for e in self.edges:
3      adj[e["src"]].append(e["dst"])
4
5  visited = set()
6  queue = deque([self.input_id])
7  while queue:
8      node = queue.popleft()
9      if node in visited:
10         continue
11     visited.add(node)
12     for nxt in adj[node]:
13         queue.append(nxt)
14
15  for node_id, info in self.nodes.items():
16     if node_id not in visited and info["type"] !=
17         ↳ "__input__":
18         error(f"Node '{node_id}' is not reachable from
19             ↳ input.")
20
21  if self.output_id not in visited:
22     error(f"Output node '{self.output_id}' is not
23         ↳ reachable.")

```

this easily catches disconnected subgraphs or broken paths from input to output

cycle detection and reachability are totally separate checks. a graph can be acyclic but fail reachability if things are disconnected, or it can be connected but have cycles. a valid neural network DAG needs both

4.3 Code Generator

Definition 4.3.1 ▶ CodeGenerator

the `CodeGenerator` class inside `code_generator.py` has four main steps:

1. `_compute_graph()`: sets up adjacency lists and does the top-sort
2. `_assign_vars()`: picks a Python variable name for every node
3. `_emit_init()` + `_emit_forward()`: generates the actual lines of code
4. `_assemble()`: glues everything into the final Python string

4.4 Variable Assignment Algorithm

Definition 4.4.1 ▶ Core variable assignment invariant

we only set `var_at[n] = 'x'` when `out_degree[predecessor] == 1`
 if a node fans out to multiple spots (`out_degree > 1`), its tensor is still needed by other branches. if we overwrite `x` there, it corrupts the data. instead, the downstream node just uses its own ID as the variable name

here are all four rules we use in `_assign_vars`:

Listing 4.4.2 ▶ Variable assignment rules in `_assign_vars`

```

1 Rule 1 - Input node:
2     var_at[input_id] = 'x'
3
4 Rule 2 - Merge node (in_degree > 1):
5     var_at[n] = 'x'           if out_degree[n] <= 1
6     var_at[n] = node_id      if out_degree[n] > 1
7
8 Rule 3 - Branch start (predecessor.out_degree > 1):
9     var_at[n] = node_id
10
11 Rule 4 - Linear continuation (predecessor.out_degree == 1):
12     var_at[n] = var_at[predecessor]
```


4.4.1 MLP Trace (Linear Chain)

Example 4.4.3 ▶ Variable trace for MLP

topology: `x -> fc1 -> relu1 -> fc2 -> relu2 -> drop -> fc3 -> out`

every single predecessor has `out_degree == 1`, so Rule 4 triggers constantly and all nodes get `var_at = 'x'`. the `forward` method just uses `x` everywhere

4.4.2 ResBlock Trace (Fan-Out Then Merge)

Example 4.4.4 ▶ Variable trace for ResBlock

`x` fans out to `conv1` and `skip` (shortcut), which means `out_degree[x] = 2`

- `conv1`: hits Rule 3 $\Rightarrow$ `var_at = 'conv1'`
- `bn1`, `relu1`, `conv2`, `bn2`: hits Rule 4 $\Rightarrow$ they all use `'conv1'`
- `skip` (Residual, `in_degree=2`): hits Rule 2, and since `out_degree=1` $\Rightarrow$ `var_at = 'x'`

generated code looks like this:

Listing 4.4.5 ▶ Generated forward pass for ResBlock

```

1 conv1 = self.conv1(x)
2 conv1 = self.bn1(conv1)
3 conv1 = self.relu1(conv1)
4 conv1 = self.conv2(conv1)
5 conv1 = self.bn2(conv1)
6 x     = conv1 + x      # Residual node
7 x     = self.flatten(x)

```

4.4.3 InceptionBlock Trace (Two Parallel Branches)

Example 4.4.6 ▶ Variable trace for InceptionBlock

we have `out_degree[x] = 2` (feeding into `convA1` and `convB1`)

- `convA1`: Rule 3 $\Rightarrow$ '`convA1`'
- `convB1`: Rule 3 $\Rightarrow$ '`convB1`'
- `convA2`: Rule 4 $\Rightarrow$ carries over '`convA1`'
- `convB2`: Rule 4 $\Rightarrow$ carries over '`convB1`'
- `cat1` (Concat, `in_degree=2`): hits Rule 2 $\Rightarrow$ '`x`'

generated code:

Listing 4.4.7 ▶ Generated forward pass for InceptionBlock

```

1 convA1 = self.convA1(x)
2 convA1 = self.convA2(convA1)
3 convB1 = self.convB1(x)
4 convB1 = self.convB2(convB1)
5 x      = torch.cat([convA1, convB1], dim=1)  # Concat
        ↪ node
6 x      = self.bn(x)
7 x      = self.out(x)

```

4.4.4 TransformerEncoder Trace (Two Implicit Residuals)

Example 4.4.8 ▶ Variable trace for TransformerEncoder

here `out_degree[x] = 2` and `out_degree[norm1] = 2`

- `attn`: Rule 3 $\Rightarrow$ '`attn`'
- `drop1`: Rule 4 $\Rightarrow$ '`attn`'
- `norm1` (`in_deg=2`, `out_deg=2`): hits Rule 2 $\Rightarrow$ '`norm1`'
- `ff1`: Rule 3 $\Rightarrow$ '`ff1`'
- `gelu1`, `drop2`, `ff2`: Rule 4 $\Rightarrow$ '`ff1`'
- `norm2` (`in_deg=2`, `out_deg=1`): hits Rule 2 $\Rightarrow$ '`x`'

generated code:

Listing 4.4.9 ▶ Generated forward pass for TransformerEncoder

```

1  attn, _ = self.attn(x, x, x)    # MultiHeadAttn
   ↪ triple-pass
2  attn      = self.drop1(attn)
3  norm1     = self.norm1(x + attn) # implicit residual_1
4  ff1       = self.ff1(norm1)
5  ff1       = self.gelu1(ff1)
6  ff1       = self.drop2(ff1)
7  ff1       = self.ff2(ff1)
8  x         = self.norm2(norm1 + ff1) # implicit
   ↪ residual_2
9  x         = self.out(x)
10 x         = self.flatten(x)
11 return x

```

4.4.5 Special Code Generation Cases

Pattern	Condition	Generated Code
MultiHeadAttn	Layer type	<code>out, _ = self.attn(x, x, x)</code>
LSTM / GRU	Layer type	<code>out, _ = self.lstm(x)</code>
Residual	No module	<code>out = main + shortcut</code>
Add	No module	<code>out = a + b</code>
Concat	No module	<code>out = torch.cat([a,b], dim=d)</code>
Split	No module	<code>split_0, split_1, ... = torch.chunk(x,n,dim=d)</code>
Implicit residual	<code>residual_*</code> label	<code>out = self.norm(skip + main)</code>

parameter name mapping: we use shorter names in the DSL, so the code generator automatically maps them to what PyTorch expects: `in_ch` → `in_channels`, `out_ch` → `out_channels`, `kernel` → `kernel_size`

4.4.6 Output Code Structure

Listing 4.4.10 ▶ Generated output code template

```

1  import torch
2  import torch.nn as nn
3
4
5  class <ModelName>(nn.Module):
6      def __init__(self):
7          super(<ModelName>, self).__init__()
8          # one line per parameterized layer node
9
10     def forward(self, x):
11         # one line per node in topological order
12         return <output_var>
13
14
15 if __name__ == '__main__':
16     device = torch.device('<device>')
17     model  = <ModelName>().to(device)
18     x      = torch.randn(<batch_size>,
19         ↪ <input_shape>).to(device)
20     print(model(x).shape)

```

4.5 Shape Inference

Definition 4.5.1 ▶ Shape Inference

the `shape_inference.py` module grabs tensor shapes from the input node and pushes them forward through the topological order. each layer type has specific math rules applied to it: Linear changes the last dimension, Conv2d does the math for spatial dimensions, Flatten squashes dimensions together, and so on

if it catches a dimension mismatch, it throws a **ShapeError** telling you the exact line number where the node was declared and hard-stops the compilation. if you use the

`--show-shapes` flag, it prints out the output shape for every node

4.6 Graphviz Visualization

we use the `graph_viz.py` module to dump Graphviz DOT files. it color-codes the nodes by layer type (green for input, blue for Linear, orange for Conv, and purple for Normalization). the output node gets double lines (`peripheries=2`) and labeled edges show up as dashed lines. if you have shape inference turned on, it writes the shapes right onto the nodes

Listing 4.6.1 ▸ Generating and rendering a graph

```
1 python main.py -i input/inception.nng --graph-viz
   ↪ output/inception.dot
2 dot -Tpng output/inception.dot -o output/inception.png
```

4.7 Training Scaffold

if you specify a `loss` in the `config` block, the code generator writes a full training loop for you, complete with loss function, optimizer, and the epoch loop. we support a bunch of loss functions like `CrossEntropyLoss`, `MSELoss`, `BCELoss`, `BCEWithLogitsLoss`, `L1Loss`, `NLLLoss`, and `SmoothL1Loss`, plus optimizers like `Adam`, `SGD`, `AdamW`, and `RMSprop`

4.8 Quantization Annotations

you can drop the `quant` parameter on literally any node. we handle three modes: `"dynamic"`, `"static"`, and `"qat"`. for QAT nodes, we wrap them in `torch.quantization.QuantWrapper`. if even one node has a quantization annotation, we automatically inject `QuantStub` and `DeQuantStub` into `__init__` and `forward`

4.9 ONNX Export

our `onnx_export.py` module takes the generated code, instantiates the model, and runs `torch.onnx.export` to give you an `.onnx` file with a dynamic batch axis and opset 17

Listing 4.9.1 ▸ Exporting a model to ONNX

```
1 python main.py -i input/mlp.nng --onnx output/mlp.onnx
```

Evaluation

5.1 Test Suite

Definition 5.1.1 ▶ Test statistics

- total tests: **55**
- framework: **pytest** version 8.4.2
- runtime: around 1.09 seconds

File	Tests	Coverage
<code>test_parser.py</code>	8	grammar parses all 4 examples with zero syntax errors; config, label, shape
<code>test_listener.py</code>	14	4 valid inputs; 10 error conditions with SystemExit
<code>test_codegen.py</code>	19	nn class names, parameter values, forward patterns
<code>test_e2e.py</code>	14	full compile + instantiate + forward pass with correct shape

how to run the tests:

Listing 5.1.2 ▶ Running the test suite

```
1 python -m pytest tests/ -v
```

5.2 Forward Pass Results

Example 5.2.1 ▶ MLP

- input: `tensor(784)` — output: `(batch, 10)`
- test: making sure Softmax rows sum to 1 (`atol=1e-5`)

Example 5.2.2 ▶ ResBlock

- input: `tensor(64, 32, 32)` — output: `(batch, 65536)`
- test: checking for the right shape and making sure the output is deterministic in `eval` mode

Example 5.2.3 ▶ InceptionBlock

- input: `tensor(32, 56, 56)` — output: `(batch, 48, 56, 56)`
- branch A: 16 channels; branch B: 32 channels. when they merge via `Concat(dim=1)`, we get 48 channels

Example 5.2.4 ▶ TransformerEncoder

- input: `tensor(512, 128)` — output: `(1024, 128)`
- has two implicit residual sub-layers without needing explicit `Residual()` nodes
- uses `MultiHeadAttn` with `embed_dim=128, num_heads=8, batch_first=True`

5.3 Variable Correctness Proof

Definition 5.3.1 ▶ Tensor non-overwrite proof

for every branching example, we traced the input tensor's `data_ptr()` right through the forward computation:

- **ResBlock:** `x.data_ptr()` stays exactly the same at the point we do `x = conv1 + x`
- **Inception:** `x.data_ptr()` is identical when calling both `convA1(x)` and `convB1(x)`
- **Transformer:** `x.data_ptr()` is untouched when `norm1(x + attn)` gets

called; `norm1.data_ptr()` is untouched when `norm2(norm1 + ff1)` gets called

this works perfectly because of our invariant: `var_at[n]='x'` only triggers when `out_degree=1`

5.4 Codebase Statistics

File/Directory	Lines
<code>NNGraph.g4</code>	56
<code>custom_listener.py</code>	303
<code>code_generator.py</code>	327
<code>shape_inference.py</code>	193
<code>graph_viz.py</code>	90
<code>onnx_export.py</code>	47
<code>main.py</code>	121
<code>tests/</code> (4 files)	575
<code>required_code_collection/</code>	≈100
Total	≈1812

- language: Python 3.14.3
- ANTLR: 4.13.2
- PyTorch: 2.10.0
- pytest: 8.4.2

Design Decisions

6.1 Listener vs. Visitor

we chose the listener pattern because it naturally handles state accumulation. the `exit*` methods let us directly mess with `self.nodes` and `self.edges` without having to pass values all the way back up the tree

6.2 `load_from_listener()` vs. `generate(traversal)`

some compilers use `CodeGenerator.generate(traversal)` which gets a post-order token list, which is great if you're writing stack-based bytecode. but since we're building a graph compiler, it makes way more sense to feed it the graph structure directly (nodes + edges) instead of a linear token stream. so we just use `load_from_listener(listener)`

6.3 Variable Naming with `node_id`

the language specs show single-letter or descriptive names in the examples. we decided to just use the actual node ID as the variable name because:

- it's totally deterministic and scales nicely for huge graphs without needing a messy counter
- it's way easier to read—seeing `convA1` makes a lot more sense than just `a`
- the underlying math ends up being exactly identical anyway

Conclusion and Future Work

7.1 Conclusion

`nngraph dsl` gives you a solid six-stage compiler pipeline to turn your `.nng` file into a fully working `nn.Module`. we built it with an ANTLR4 grammar packing 14 parser rules, 24 layer types, 8 validation checks, and a really smart variable assignment algorithm that handles messy branching without accidentally overwriting tensors. we've validated the whole thing with 55 tests running against four different architectural examples

7.2 Implemented Features

Feature	Description
Shape inference	tensor shape propagation and mismatch detection at compile time (<code>--show-shapes</code>)
Graphviz visualization	model graph output in DOT format (<code>--graph-viz</code>)
Training scaffold	training loop generation with optimizer and loss from <code>config</code> block
Quantization	<code>quant</code> metadata for model quantization conversion
ONNX export	model export to ONNX format (<code>--onnx</code>)

7.3 Future Work

Feature	Description
Named sub-graphs	reusable macro blocks
Dynamic control flow	<code>if/else</code> branches within <code>forward</code>

References

1. Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf
2. Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS 2019*
3. NADER Framework: Neural Architecture Design via Representation (referenced in `NNGraph_DSL_Specification.md`)
4. Zhang, A. M. *amznotes LaTeX template*. github.com/alexmingzhang/latex-notes-template